

Technical Project Proposal and Architecture Report: AP Government Information Intelligence Platform

Executive Summary and Problem Statement

The public sector functions as a massive generator of critical, highly structured, and semi-structured data spanning legislation, budgets, departmental directives, and public procurement. In the state of Andhra Pradesh (AP), this information is predominantly published in the form of Government Orders (GOs), budget releases, schematic guidelines, and tender documents. Following a three-year blackout, the AP Government Orders Issue Register (GOIR) portal was restored in August 2024 via G.O. MS No. 93, bringing a vast historical repository of GOs dating from 2008 back into the public domain¹. This repository encapsulates vital administrative data, ranging from the implementation guidelines for the THALLIKI VANDANAM educational scheme to granular public service organizational regulations and pension disbursement guidelines³. Simultaneously, separate portals dedicated to AP e-procurement distribute tender schedules, infrastructure agreements, and budget allocations⁵.

Despite the renewed availability of this information, the data architecture surrounding it remains highly fragmented, obfuscated by departmental silos, and heterogeneous in format. A significant portion of these documents exists as unstructured or semi-structured Portable Document Formats (PDFs). While modern resolutions are digitally native, historical archives often comprise severely degraded, scanned physical copies exhibiting misaligned orientations and noise¹. Furthermore, the linguistic corpus is inherently bilingual, containing complex intersections of English and Telugu, often punctuated by administrative domain-specific colloquialisms, numeric tables, and unique bureaucratic abbreviations.

This fragmentation presents a profound operational and transparency bottleneck. Bureaucrats, journalists, and citizens routinely spend excessive time manually parsing through legacy portals, downloading disparate PDFs, and attempting to synthesize historical context across multiple departments. Standard keyword search engines deployed on these legacy platforms are structurally incapable of executing semantic reasoning or answering complex, multi-document queries.

The objective of the "AP Government Information Intelligence Platform" is to architect and deploy a production-grade, evidence-grounded Artificial Intelligence search and synthesis engine. This system will ingest, parse, and structure fragmented AP government data into a unified, queryable intelligence layer. To achieve this, the platform will utilize an advanced Retrieval-Augmented Generation (RAG) architecture powered by a sophisticated multi-stage retrieval pipeline. This pipeline encompasses Hybrid Retrieval—combining traditional SQL relational filtering, BM25 sparse keyword search, and dense semantic vector search—followed by a highly precise Cross-Encoder Reranker⁹.

This project operates under strict, consumer-grade hardware constraints: the system must be

highly optimized for local execution on a laptop equipped with 32 GB of unified Random Access Memory (RAM) and an Intel Iris Xe integrated Graphics Processing Unit (iGPU), lacking a dedicated NVIDIA tensor accelerator¹¹. Consequently, the architecture demands aggressive software engineering and optimization strategies. These include memory-mapped embedding execution, hierarchical Parent-Child chunking to preserve context, and intelligent hardware-specific backend compilation utilizing Intel's SYCL and OpenVINO toolkits¹³. A critical mandate of this intelligence platform is the complete elimination of algorithmic hallucinations. Every assertion synthesized by the underlying Large Language Model (LLM) must be strictly grounded by traceable citations, returning exact source Uniform Resource Locators (URLs), document titles, and page numbers. This proposal outlines the end-to-end systems engineering required to transition this concept from raw ingestion to a benchmarked, high-throughput, hallucination-free intelligence platform.

System Architecture Design

The architectural topology is designed as a directed acyclic graph of data operations, progressing from automated ingestion to final user interface text streaming. The pipeline guarantees zero data loss, strict provenance tracking, and high-fidelity retrieval across bilingual text.

Data Ingestion and Document Parsing Pipeline

The foundation of the intelligence platform relies on the automated extraction of data from decentralized governmental endpoints, specifically the GOIR and e-procurement sites⁷. The ingestion layer employs asynchronous web crawlers leveraging Python's httpx and BeautifulSoup4 libraries to mirror the repositories. During this initial pass, the crawlers extract baseline HTML metadata, including the issuing Department, GO Number, Issue Date, and Subject classification.

Because the raw corpus contains non-searchable scanned PDFs, standard heuristic text extraction libraries are fundamentally insufficient. The architecture mandates the integration of PaddleOCR, specifically the PP-OCRV5 and PP-StructureV3 pipelines¹⁶. PaddleOCR is selected for its robust, state-of-the-art multilingual text spotting capabilities, which natively support both English and Indic scripts, including Telugu, without requiring inefficient model-switching during inference¹⁷.

The ingestion workflow executes a deterministic sequence of computer vision tasks. Initially, PP-StructureV3 performs layout analysis, classifying document regions into 23 distinct categories such as document titles, abstract text, tables, and footnotes¹⁹. This structural awareness is critical for preventing the amalgamation of unrelated textual blocks. Following layout detection, structured tables within budget documents and tender schedules are routed through the SLANet table recognition architecture, translating visual grids directly into Markdown or JSON formats¹⁶. This preserves the row-column relationships essential for downstream LLM reasoning. Finally, misoriented text lines are corrected by an orientation classification model, and the PP-HGNetV2 backbone executes character recognition¹⁶. The extracted text is then enriched with temporal metadata and mapped to a standardized JSON schema, ready for vectorization.

Feature	PyPDF2 / pdfminer	Tesseract OCR	PaddleOCR (PP-StructureV3)
Input Support	Digitally native PDFs only	Scanned images and PDFs	Scanned images, PDFs, complex layouts
Language Support	Relies on embedded fonts	Moderate Indic script accuracy	High-fidelity bilingual (English/Telugu) ¹⁷
Layout Analysis	None	Basic bounding boxes	Advanced 23-class region detection ¹⁹
Table Extraction	Fails on grids	Returns garbled text streams	Native conversion to Markdown/JSON ¹⁷
Hardware Acceleration	CPU only	CPU only	Intel OpenVINO / CPU / GPU ¹⁶

Hierarchical Parent-Child Chunking Strategy

Traditional RAG architectures suffer from the "Lost in the Middle" phenomenon and severe context fragmentation when relying on naive, fixed-size token splitting²⁰. If chunks are arbitrarily small, the LLM receives disjointed sentences and lacks the surrounding narrative to generate a coherent answer. Conversely, if chunks are exceedingly large, the semantic density is diluted across multiple topics, significantly reducing the accuracy of the vector similarity search during the retrieval phase²².

To resolve this dichotomy, the architecture implements Hierarchical Parent-Child Chunking, also known as Small-to-Big Retrieval²². The parsed Markdown text from the OCR pipeline is divided into two distinct relational tiers. The "Parent Chunks" represent large, logical sections of a document—typically 1,024 to 2,048 tokens in length, corresponding to a full legislative section or document page. These parent chunks are stored in a relational document store but are not embedded into the vector space²³.

Subsequently, each parent chunk is subdivided into smaller, highly focused "Child Chunks," typically ranging from 256 to 512 tokens, with a 10-20% sliding window overlap to prevent sentence bisection²³. Only these child chunks are passed through the embedding model to be stored in the vector database. During query execution, the semantic search operates exclusively on the high-precision child chunks. When a relevant child chunk is identified via vector similarity, the system retrieves its associated parent chunk via a foreign key relationship

and injects the broader context into the LLM prompt²⁴. This ensures pinpoint semantic matching without sacrificing the narrative context necessary for accurate, hallucination-free synthesis.

Database Tier: PostgreSQL and pgvector

Rather than introducing network latency and operational complexity by deploying disparate NoSQL document stores alongside standalone vector databases (e.g., Pinecone or ChromaDB), this architecture centralizes state management entirely within PostgreSQL, augmented by the pgvector extension²⁶. PostgreSQL ensures ACID compliance, enables complex relational JOIN operations between GO metadata and text embeddings, and provides a robust, unified backup mechanism⁹.

The database schema strictly separates the document taxonomy from the vector embeddings to optimize memory paging. A documents table stores document-level metadata such as the GO type, issuing department, publication date, and the original source URL. A parent_chunks table stores the large text blocks required for LLM synthesis. Finally, a child_chunks table stores the exact extracted text segments alongside a high-dimensional vector column containing the embeddings.

To ensure strict data boundaries and support future multi-tenant capabilities—such as segregating the Department of Finance records from Municipal Administration archives based on user clearance—the database leverages PostgreSQL Row Level Security (RLS)²⁸. RLS allows the database to enforce filtering directly at the kernel level, ensuring that user queries only retrieve vectors associated with permitted administrative zones³⁰. Policies are defined using the CREATE POLICY directive combined with application-injected session variables (e.g., current_setting('app.current_tenant'))²⁹. This provides a zero-trust data retrieval layer, guaranteeing that even if the application routing logic fails, the database kernel will mathematically reject unauthorized vector scans.

Multi-Stage Retrieval: Hybrid Search and Cross-Encoder Reranking

Relying solely on dense semantic vector search often fails catastrophically when users query specific, domain-heavy identifiers such as "G.O.Ms.No. 57" or exact 16-digit tender alphanumeric codes³. Dense embeddings capture conceptual proximity but often exhibit "semantic blindness" to exact lexical overlaps. Therefore, the retrieval engine implements a sophisticated three-stage pipeline.

The first stage executes Hybrid Retrieval. The user's query is initially parsed to extract explicit metadata filters (e.g., "Department of Health", "Year 2025"), injecting these as SQL WHERE clauses to aggressively narrow the search space before any distance calculations occur. Simultaneously, the query undergoes dense semantic search against the pgvector index to identify conceptual matches, alongside a traditional term-frequency/inverse document-frequency (TF-IDF) scoring via a BM25 sparse index to capture exact keyword overlaps and rare administrative acronyms¹⁰.

To reconcile the disparate scoring mechanisms of BM25 (which outputs unbounded term frequency scores) and dense vector search (which outputs cosine similarity bounded between -1 and 1), the second stage utilizes Reciprocal Rank Fusion (RRF)³⁵. RRF is a rank aggregation

algorithm that mathematically rewards documents appearing near the top of multiple independent retrieval lists without requiring complex score normalization³⁷. The formulation applied is:

$$RRF(d) = \sum_{r \in R} \frac{1}{k + r(d)}$$

Here, d represents a retrieved document chunk, R is the set of retrieval rankers (BM25 and Dense), $r(d)$ is the specific rank of document d within a ranker's list, and k is a smoothing constant strictly set to 60³⁵. This constant is empirically proven to balance the influence of top-ranked and lower-ranked items while gracefully handling tie-breaking scenarios across heterogeneous retrieval models³⁵.

While RRF effectively identifies a broad pool of highly likely candidates, bi-encoder embedding models are inherently limited because they encode the query and the document entirely separately, compressing them into fixed-length vectors before comparison. To resolve this, the

third and final stage introduces a Cross-Encoder Reranker¹⁰. The reranker takes the top N candidate chunks from the RRF output and processes the query and each chunk simultaneously through a cross-attention transformer network. Because the attention heads can directly compare the tokens of the query against the tokens of the document, the cross-encoder outputs a highly accurate, calibrated relevance logit. This computationally

expensive step is restricted strictly to the top N candidates, ensuring maximum precision without inducing unacceptable latency overheads.

LLM Synthesis and Server-Sent Events (SSE) Streaming

The final K parent chunks, identified by mapping the reranked child chunks back to their relational parents, are assembled into a robust, augmented prompt. A strict metaprompting framework is enforced at this juncture. The LLM is instructed via system-level parameters to construct its response relying exclusively on the provided context, categorically rejecting external knowledge to prevent hallucination. Furthermore, the metaprompt strictly mandates the appendage of inline citations, formatted explicitly as [Document Title, Page X], to every factual claim synthesized from the context.

To provide a low-latency, highly responsive user experience, the frontend interface receives the LLM response via Server-Sent Events (SSE)³⁹. Unlike WebSockets, which require the complex overhead of persistent bidirectional state management, SSE is a unidirectional protocol utilizing chunked transfer encoding over standard HTTP⁴¹. This architectural choice perfectly aligns with the autoregressive, token-by-token generation nature of LLMs. As the local model computes the response, tokens are pushed to the browser in real-time over a single, multiplexed HTTP connection, entirely eliminating blank-screen latency and significantly improving the perceived

performance of the platform³⁹.

Feature	HTTP Polling	WebSockets	Server-Sent Events (SSE)
Connection Type	Client-initiated loops	Bidirectional, persistent	Unidirectional (Server to Client) ⁴²
Protocol	Standard HTTP	ws:// or wss://	Standard HTTP / HTTP2 ⁴⁴
Complexity	Low	High (requires custom routing)	Low (native browser API support) ⁴⁰
LLM Streaming Suitability	Poor (high latency)	Overkill (client rarely sends data)	Optimal (designed for server push) ³⁹

Hardware Optimization Strategy

A defining engineering constraint of this project is the strict adherence to a specific consumer-grade hardware profile: a laptop with 32 GB of unified RAM and an Intel Iris Xe integrated GPU. Unoptimized Python deep learning pipelines naturally default to Central Processing Unit (CPU) execution on such hardware, resulting in unacceptable inference bottlenecks. The strategy requires precision engineering across the execution providers to maximize the utilization of the Intel architecture.

Embedding Model Optimization with OpenVINO

Generating dense embeddings for thousands of governmental documents is computationally intensive. The architecture utilizes the BGE-M3 (Multi-lingual, Multi-functionality, Multi-granularity) embedding model⁴⁵. BGE-M3 is uniquely suited for this pipeline because it natively understands cross-lingual alignments between English and Indian languages (including Telugu) and produces dense vectors highly optimized for cosine similarity comparisons¹⁰. To accelerate BGE-M3 on the Intel Iris Xe iGPU, the model is executed using the ONNX Runtime via the OpenVINOExecutionProvider⁴⁸. OpenVINO is Intel's proprietary toolkit designed to map deep learning computational graphs directly to the execution units of Intel hardware, bypassing standard, unoptimized instruction sets¹⁴.

A significant challenge with local deployment is the "first inference latency." Deep learning models suffer from significant loading times as kernels compile dynamically for the specific hardware target upon initialization. By enabling the `cl_cache` API within the OpenVINO configuration, compiled binary representations of OpenCL kernels are cached directly to the host disk¹². Subsequent executions bypass the Just-In-Time (JIT) recompilation, drastically

reducing application startup latency. Furthermore, the embedding weights are quantized to FP16. This takes advantage of the native half-precision support on Intel iGPUs, effectively doubling memory throughput and compute bandwidth with near-zero accuracy degradation compared to standard FP32 execution⁵¹.

pgvector Indexing Strategy: HNSW vs. IVFFlat

Operating a vector database within a 32 GB RAM constraint necessitates highly deliberate index selection. The pgvector extension offers two primary approximate nearest neighbor (ANN) indexes: HNSW (Hierarchical Navigable Small World) and IVFFlat (Inverted File with Flat Compression)⁵³.

Feature	HNSW	IVFFlat
Algorithmic Approach	Multi-layered proximity graph ⁹	K-means Voronoi cell partitioning ⁵³
Recall / Accuracy	95%+ out-of-the-box consistency ⁵³	70-80% without heavy tuning ⁵⁴
Build Time	Slower; complex edge computation ⁹	Faster; simple centroid calculation ⁵⁶
Memory Footprint	High; stores graph edges at every node ⁵⁴	Low; highly compact list structures ⁵⁴
Incremental Inserts	Excellent; dynamically adapts to new data ⁵⁴	Poor; recall drifts as boundaries shift ⁵³

Given the dynamic nature of government data—characterized by a continuous, daily influx of newly published GOs and tender schedules—HNSW is the unequivocally superior choice despite its larger RAM footprint³. IVFFlat calculates its cluster centroids at build time; as new documents are inserted, the clusters become misaligned with the actual data distribution, leading to severe recall degradation unless the index is periodically rebuilt⁵³. HNSW, conversely, incrementally updates its graph topology, absorbing continuous writes without maintenance windows⁵⁴. For datasets under 10 million vectors, the 32 GB system RAM is more than sufficient to hold the HNSW graph entirely in memory⁵³.

The index is constructed using the vector_cosine_ops distance function, utilizing carefully optimized build parameters: m = 16 (the maximum number of bidirectional links per node) and ef_construction = 64 (the size of the dynamic candidate list during the graph construction phase)⁹. At runtime, the hnsw.ef_search variable is dynamically tuned between 40 and 100 on a per-query basis, allowing the application layer to balance query latency against mathematical recall depending on the complexity of the user's prompt⁹.

LLM Execution via llama.cpp and SYCL

Running a generative Large Language Model locally for RAG synthesis on integrated graphics requires bypassing heavy, overhead-laden frameworks like PyTorch. Instead, the architecture utilizes the highly optimized C++ inference engine, llama.cpp¹¹.

The Intel Iris Xe shares memory with the system via a unified architecture. While 32 GB of capacity is generous, the memory bandwidth (typically limited to 50-70 GB/s on DDR4/DDR5 laptop configurations) acts as a severe, unavoidable bottleneck for autoregressive token generation⁶⁰. Loading a standard 8-billion parameter model at 16-bit precision will rapidly saturate this bandwidth, triggering operating system swap thrashing and reducing generation speeds to an unusable crawl of under 2 tokens per second¹¹.

To circumvent this hardware limitation, a three-pronged strategy is deployed:

1. **Model Selection and Quantization:** The system deploys highly efficient, sub-4-billion parameter models, specifically Microsoft's Phi-4-Mini (3.8B), quantized to 4-bit block formats utilizing the Q4_K_M GGUF specification¹¹. This limits the model's active RAM footprint to approximately 2.5 GB¹¹, leaving ample space for the context window, the operating system, and the PostgreSQL buffers.
2. **SYCL Backend Execution:** The llama.cpp binary is compiled natively using Intel's oneAPI SYCL backend (build.sh fp16), mapping the heavy matrix multiplications directly to the Xe Execution Units rather than falling back to the generic CPU instructions¹³.
3. **KV Cache Management:** The context length is strictly capped at 4,096 tokens. Crucially, the Key-Value (KV) cache is quantized to 8-bit (q8_0) precision to prevent out-of-memory errors and bandwidth saturation during long-context document synthesis¹¹.

Component	Estimated Memory Allocation	Purpose
Operating System	~4.0 GB	Core system processes and background tasks
PostgreSQL & pgvector	~6.0 GB	Database buffers, HNSW graph, RLS execution
OpenVINO (BGE-M3 + Reranker)	~3.5 GB	Embedding generation and Cross-Encoder weights
Phi-4-Mini (Q4_K_M GGUF)	~2.5 GB	LLM weights mapped to unified memory ¹¹

KV Cache (q8_0 quantized)	~2.0 GB	Attention mechanism memory for 4k context ⁶²
Available Headroom	~14.0 GB	Application overhead, OCR burst processing, cache

If the local processing pipeline cannot meet the acceptable throughput target (e.g., requiring >15 tokens/second for exceptionally complex comparative queries), the architecture includes an abstract fallback routing mechanism. This router seamlessly shifts inference to lightweight, low-cost API models (e.g., Google's Gemini 1.5 Flash or OpenRouter endpoints)¹⁰, treating the local SYCL-backed LLM as an offline, privacy-preserving fail-safe.

Technology Stack Justification

The selection of the technology stack represents a rigorous optimization for accuracy, multi-lingual capability, and stability within low-resource environments.

Data Extraction: PaddleOCR vs. Tesseract

While Tesseract remains the open-source standard for Western languages, its accuracy degrades significantly when processing low-quality Indic script documents, rendering it inadequate for the AP government corpus which heavily features Telugu⁴⁵. PaddleOCR (PP-OCRv6) provides native, unified recognition for 109 languages without requiring complex model switching¹⁷. More critically, its layout analysis toolkit, PP-StructureV3, accurately parses multi-column PDF structures and extracts complex budget tables into relational Markdown formats, a prerequisite for preventing garbled text blocks from corrupting the embedding space¹⁶.

Database and Search: PostgreSQL and FastAPI

PostgreSQL with pgvector was selected over dedicated vector databases to eliminate network latency, reduce the microservice footprint, and leverage ACID compliance. Furthermore, PostgreSQL allows for exact date and departmental filtering via standard SQL WHERE clauses executed simultaneously with the dense vector scan, enabling true, performant hybrid searches within a single query⁹.

The backend orchestration layer is constructed using FastAPI (Python). FastAPI natively supports ASGI asynchronous execution, which is vital for the non-blocking I/O required when simultaneously orchestrating SQL execution, ONNX embedding generation, and LLM inference. FastAPI's built-in StreamingResponse integrates flawlessly with Server-Sent Events (SSE), streaming generated text tokens to the frontend instantaneously⁴³.

Frontend: React

The frontend user interface is developed using React. React's component-driven architecture is highly suited for building complex search interfaces with dynamic filtering toggles. Furthermore, React seamlessly consumes SSE streams using standard browser EventSource APIs, enabling smooth, typewriter-style rendering of the LLM output alongside dynamically updating citation links⁴⁰.

8-Week Implementation Roadmap

The project is structured into four two-week sprints, adhering to rigorous software engineering principles, moving from foundational infrastructure to iterative optimization and final benchmarking.

Timeline	Phase	Key Deliverables and Objectives
Week 1-2	Data Ingestion & OCR Pipeline	Deploy asynchronous crawlers against goir.ap.gov.in and e-procurement nodes. Setup the PaddleOCR (PP-StructureV3) environment. Process a 5,000-document pilot dataset, focusing on degraded bilingual PDFs. Establish automated table-to-Markdown extraction routines.
Week 3-4	Chunking & Vector Database	Implement Python-based Recursive Character Text Splitters. Develop the relational Parent-Child Chunking algorithm (256 child / 1024 parent tokens) in PostgreSQL. Compile BGE-M3 with OpenVINOExecutionProvider for the Intel Iris Xe iGPU. Configure HNSW indexes (vector_cosine_ops, m=16, ef_construction=64).
Week 5-6	Hybrid Search & Reranking	Build the SQL metadata filter layer and generate BM25 sparse indexes. Implement the Reciprocal Rank Fusion (RRF) scoring

		algorithm (k=60). Integrate the Cross-Encoder Reranker for top-N candidate refinement. Expose retrieval via FastAPI endpoints.
Week 7-8	LLM Synthesis & UI Delivery	Compile llama.cpp with the Intel oneAPI (SYCL) backend. Deploy the Phi-4-Mini-Q4_K_M GGUF model with an 8-bit KV cache. Develop the metaprompt enforcing strict [Citation] behaviors. Build the React frontend with SSE architecture. Execute final metric evaluations.

Quantitative Evaluation Methodology

A production-grade intelligence pipeline requires objective, quantitative validation to ensure that architectural choices yield empirical improvements rather than anecdotal observations. The system is evaluated across three independent vectors: Ingestion accuracy, Retrieval fidelity, and Synthesis quality.

OCR and Ingestion Benchmarking

The extraction accuracy of PaddleOCR on the bilingual AP dataset is measured utilizing Character Error Rate (CER) and Word Error Rate (WER)⁶⁵. A ground-truth baseline of 100 manually transcribed AP GOs (50 English, 50 Telugu) serves as the benchmark for the PP-OCRv5 engine, ensuring that character degradation does not corrupt downstream embeddings.

Retrieval Optimization (MRR and Recall@K)

To quantify the effectiveness of the Hybrid Search, RRF, and Cross-Encoder parameters, a dataset of 200 synthetic queries generated from actual administrative scenarios (e.g., "What is the budget allocation for the THALLIKI VANDANAM scheme in 2025-26?"³) is evaluated.

- **Recall@K:** Measures the percentage of queries where the exact correct chunk appears in the top K retrieved results⁶⁶. The architectural target is >90% Recall@5.
- **Mean Reciprocal Rank (MRR):** Evaluates the average position of the first relevant

document across all test queries, ensuring that the Cross-Encoder pushes the most relevant context to rank 1.

RAG Synthesis Evaluation (Ragas Framework)

The synthesis quality is autonomously benchmarked utilizing the Ragas (Retrieval Augmented Generation Assessment) framework⁶⁷. Ragas leverages an "LLM-as-a-judge" mechanism to compute critical metrics mathematically:

- **Context Precision:** A ratio determining if the retrieved chunks directly address the prompt, severely penalizing the retrieval of irrelevant noise⁶⁷.
- **Context Recall:** Evaluates whether all necessary information required to completely answer the query was successfully retrieved from the database⁶⁸.
- **Faithfulness (Groundedness):** Strictly scores the LLM's output against the provided context. If the LLM generates a numerical claim regarding AP budget figures that is not explicitly present in the retrieved parent chunk, it is programmatically flagged as a hallucination⁶⁸.
- **Citation Accuracy:** A custom regex script parses the generated response to verify that every appended citation maps perfectly to the foreign key of the retrieved parent chunk, demanding a 100% trace rate for production deployment.

Hardware Throughput Benchmarks

Local execution constraints are monitored continuously to validate the SYCL and OpenVINO integrations.

- **Time-to-First-Token (TTFT):** Must remain under 1,500 milliseconds, heavily assisted by OpenVINO `cl_cache` disk reads.
- **Generation Throughput:** Must exceed 10 tokens/second during local llama.cpp SYCL generation on the Intel Iris Xe to ensure a fluid user experience¹¹.

Impact and Portfolio Value

The "AP Government Information Intelligence Platform" represents a profound leap beyond standard API-wrapper applications, operating squarely in the domain of deep systems engineering and applied AI research.

From an architectural standpoint, the project demonstrates a masterful handling of heterogeneous data and severe resource constraints. By executing state-of-the-art multilingual OCR on corrupted, bilingual public records, it solves a genuine accessibility gap in Indian e-governance technology.

Technically, the implementation serves as a masterclass in modern RAG pipeline engineering. The integration of Reciprocal Rank Fusion, HNSW graph tuning, and Cross-Encoder reranking translates raw theoretical mathematics into highly performant software. Implementing Small-to-Big Retrieval (Parent-Child chunking) directly attacks the "Lost in the Middle" bias of modern attention mechanisms, showcasing an understanding of AI behavior far beyond basic retrieval tutorials. Finally, by deliberately eschewing cloud GPUs in favor of pushing consumer silicon (Intel Iris Xe) to its absolute limit using SYCL wrappers, oneAPI compilations, and OpenCL caching, the project demonstrates a rigorous understanding of memory bandwidth, precision quantization, and kernel-level compute topologies.

Ultimately, this project delivers a production-grade, highly secure (PostgreSQL RLS), and highly responsive (SSE Streaming) application. It bridges the gap between fragmented bureaucratic data silos and instant, verifiable, hallucination-free public intelligence, firmly positioning the architect as a senior practitioner capable of leading end-to-end enterprise AI initiatives.

Works cited

1. A.P. government restores 'GOIR' portal - The Hindu, <https://www.thehindu.com/news/national/andhra-pradesh/ap-government-restores-goir-portal/article68806056.ece>
2. Government Orders Uploaded on Web Portal | Vijayawada News, <https://timesofindia.indiatimes.com/city/vijayawada/government-orders-uploaded-on-web-portal/articleshow/112850423.cms>
3. Latest GO's - APUTF, <https://www.aputf.org/gos/latest-gos/>
4. Andhra Pradesh GO | Principal Accountant General (A&E ... - CAG, <https://cag.gov.in/ae/andhra-pradesh/en/page-ae-andhra-pradesh-andhra-pradesh-go-s>
5. Government eProcurement System, <https://etenders.gov.in/>
6. Andhra Pradesh Tenders ,ap E- Tenders ,, <https://www.andhrapradeshtenders.com/>
7. Andhra Pradesh eProcurement: 2026 Guide to AP Tenders - Tender18, <https://tender18.com/blog-details/Andhra-Pradesh-eProcurement-Your-Guide-to-AP-Tenders-Success>
8. Private: Government Orders | Kakinada District - Andhra Pradesh, <https://kakinada.ap.gov.in/service/private-government-orders/>
9. pgvector/pgvector: Open-source vector similarity search for Postgres, <https://github.com/pgvector/pgvector>
10. IndicRAG — Multilingual Agentic Scientific RAG - GitHub, <https://github.com/DNSdecoded/IndicRAG>
11. Best Local AI Apps for Low-End PCs in 2026 (8GB RAM, No GPU), <https://www.promptquorum.com/power-local-llm/best-local-ai-app-low-end-pc>
12. OpenVINO™ Execution Provider + Model Caching = Better First, <https://medium.com/openvino-toolkit/openvino-execution-provider-model-caching-better-first-inference-latency-for-your-onnx-models-cbcef1e79d65>
13. Performance of llama.cpp on Intel GPU with SYCL backend #23313, <https://github.com/ggml-org/llama.cpp/discussions/23313>
14. OpenVINO - onnxruntime, <https://skottmckay.github.io/onnxruntime/docs/reference/execution-providers/OpenVINO-ExecutionProvider.html>
15. How to View Govt Orders Andhra Pradesh Download Govt GOs Online, <https://www.youtube.com/watch?v=0kP8QK-zBzA>
16. (PDF) PaddleOCR 3.0 Technical Report - ResearchGate, https://www.researchgate.net/publication/393511573_PaddleOCR_30_Technical_Report
17. GitHub - PaddlePaddle/PaddleOCR,

- <https://github.com/PADDLEPADDLE/PADDLEOCR>
18. PaddleOCR multilingual model training tutorial - Kaggle,
<https://www.kaggle.com/code/oralia/paddleocr-multilingual-model-training-tutorial>
 19. Layout Detection - PaddleX Documentation,
https://paddlepaddle.github.io/PaddleX/3.4/en/module_usage/tutorials/ocr_modules/layout_detection.html
 20. RAG Compression: The Missing Layer in Your AI Pipeline (Live Demo),
<https://medium.com/@visrow/rag-compression-the-missing-layer-in-your-ai-pipeline-live-demo-8b52bd742708>
 21. Overcome Lost In Middle Phenomenon In RAG Using ... - AI Planet,
<https://medium.aiplanet.com/overcome-lost-in-middle-phenomenon-in-rag-using-longcontextretriever-2334dc022f0e>
 22. Advanced RAG (Chunking, Reranking, Hybrid Search),
<https://aiengineeringfromscratch.com/lesson.html?path=phases/11-llm-engineering/07-advanced-rag>
 23. ai-engineering-guide/03-retrieval-and-rag/05-chunking-strategies,
<https://github.com/dipakkr/ai-engineering-guide/blob/main/03-retrieval-and-rag/05-chunking-strategies.md>
 24. The Beauty of Parent-Child Chunking. Graph RAG Was Too Slow for,
https://www.reddit.com/r/Rag/comments/1mtcvs7/the_beauty_of_parentchild_chunking_graph_rag_was/
 25. Parent-Child Chunking in LangChain for Advanced RAG - Medium,
<https://medium.com/@seahorse.technologies.sl/parent-child-chunking-in-langchain-for-advanced-rag-e7c37171995a>
 26. Vector Similarity Search with PostgreSQL's pgvector - A Deep Dive,
<https://severalnines.com/blog/vector-similarity-search-with-postgresqls-pgvector-a-deep-dive/>
 27. How to optimize performance when using pgvector - Azure Cosmos,
<https://learn.microsoft.com/en-us/azure/cosmos-db/postgresql/howto-optimize-performance-pgvector>
 28. Support for PostgreSQL Row Level Security (RLS) - Feature Request,
<https://community.wappler.io/t/support-for-postgresql-row-level-security-rls/47465>
 29. Documentation: 18: 5.9. Row Security Policies - PostgreSQL,
<https://www.postgresql.org/docs/current/ddl-rowsecurity.html>
 30. PostgreSQL Row-Level Security for Multi-Tenant Apps,
<https://www.openspaceservices.com/blog/software-engineering/leveraging-postgre-sql-row-level-security-in-multi-tenant-applications>
 31. How to Secure Multi-Tenant Data with Row-Level ... - OneUptime,
<https://oneuptime.com/blog/post/2026-01-25-row-level-security-postgresql/view>
 32. Multi-Tenancy using Row Level Security in Postgres - Medium,
<https://medium.com/@manishchaulagain/multi-tenancy-using-row-level-security-in-postgres-2ebfd6871539>
 33. Better RAG Results With Reciprocal Rank Fusion - MongoDB,

- <https://www.mongodb.com/resources/basics/reciprocal-rank-fusion>
34. Sparse Retriever for non-English languages : r/Rag - Reddit,
https://www.reddit.com/r/Rag/comments/1pou7qv/sparse_retriever_for_nonenglish_languages/
 35. Reciprocal Rank Fusion (RRF) Explained | Veso AI Blog,
<https://veso.ai/blog/reciprical-rank-fusion/>
 36. Reciprocal rank fusion | Elasticsearch Reference,
<https://www.elastic.co/docs/reference/elasticsearch/rest-apis/reciprocal-rank-fusion>
 37. Reciprocal Rank Fusion and Reranking for RAG | Sphere Inc.,
<https://www.sphereinc.com/blogs/reranking-and-rrf>
 38. Reciprocal Rank Fusion (RRF) explained in 4 mins - Medium,
<https://medium.com/@devalshah1619/mathematical-intuition-behind-reciprocal-rank-fusion-rrf-explained-in-2-mins-002df0cc5e2a>
 39. Streaming AI Responses in React with SSE - ScriptsHub Technologies,
<https://scriptshub.net/resources/blogs/streaming-ai-responses-react-sse/>
 40. How to Implement Server-Sent Events (SSE) in React - OneUptime,
<https://oneuptime.com/blog/post/2026-01-15-server-sent-events-sse-react/view>
 41. The Complete Guide to Streaming LLM Responses in Web,
https://dev.to/pockit_tools/the-complete-guide-to-streaming-llm-responses-in-web-applications-from-sse-to-real-time-ui-3534
 42. A short guide to setting up Server Sent Events | by Ankit K Pandey,
<https://blog.devgenius.io/a-short-guide-to-setting-up-server-sent-events-0bfaf97cc0cd>
 43. Streaming in React the Simple Way: Server-Sent Events (with FastAPI),
<https://www.youtube.com/watch?v=hOAAg1WaZh8>
 44. Understanding Server-Sent Events : Core Infrastructure for Agentic,
<https://medium.com/@balajibal/understanding-server-sent-events-core-infrastructure-for-agentic-platforms-b42f348c4789>
 45. A Multilingual Benchmark to Evaluate Question Answering capability,
<https://arxiv.org/html/2407.13522>
 46. M3-Embedding: Multi-Linguality, Multi-Functionality ... - ResearchGate,
https://www.researchgate.net/publication/384220744_M3-Embedding_Multi-Linguality_Multi-Functionality_Multi-Granularity_Text_Embeddings_Through_Self-Knowledge_Distillation
 47. Benchmarking and Building Zero-Shot Hindi Retrieval Model ... - arXiv,
<https://arxiv.org/html/2409.05401v3>
 48. FastEmbed | Haystack Documentation,
<https://docs.haystack.deepset.ai/reference/fastembed-embedders>
 49. ONNX: Train in Any Framework, Deploy on Any Hardware - DataCamp,
<https://www.datacamp.com/tutorial/onnx>
 50. OpenVINO™ is an open source toolkit for optimizing and deploying,
<https://github.com/openvinotoolkit/openvino>
 51. Intel - OpenVINO™ | onnxruntime,
<https://onnxruntime.ai/docs/execution-providers/OpenVINO-ExecutionProvider.html>

[ml](#)

52. pgvector, a guide for DBA - Part 2: Indexes (update march 2026),
<https://www.dbi-services.com/blog/pgvector-a-guide-for-dba-part-2-indexes-update-march-2026/>
53. HNSW vs IVFFlat: How to Choose the Right Vector Index,
<https://bigdataboutique.com/blog/hnsw-vs-ivfflat-how-to-choose-the-right-vector-index>
54. IVFFlat vs HNSW in pgvector: Which Index Should You Use?,
https://dev.to/philip_mccclarence_2ef9475/ivfflat-vs-hnsw-in-pgvector-which-index-should-you-use-305p
55. Optimize generative AI applications with pgvector indexing - AWS,
<https://aws.amazon.com/blogs/database/optimize-generative-ai-applications-with-pgvector-indexing-a-deep-dive-into-ivfflat-and-hnsw-techniques/>
56. Comparing IVFFlat and HNSW in pgvector: Vector Similarity,
<https://medium.com/@emreks/comparing-ivfflat-and-hnsw-with-pgvector-performance-analysis-on-diverse-datasets-e1626505bc9a>
57. Faster similarity search performance with pgvector indexes,
<https://cloud.google.com/blog/products/databases/faster-similarity-search-performance-with-pgvector-indexes>
58. IVFFlat vs HNSW in pgvector with text-embedding-3-large. When is it,
https://www.reddit.com/r/Rag/comments/1pij7q/ivfflat_vs_hnsw_in_pgvector_with/
59. getting acceleration on Intel integrated GPU/NPU : r/LocalLLaMA,
https://www.reddit.com/r/LocalLLaMA/comments/1m19iqi/getting_acceleration_on_intel_integrated_gpu_npu/
60. Intel Arc A770 Local AI Guide: 16GB VRAM, SYCL, Ollama,
<https://localaimaster.com/blog/intel-arc-a770-local-ai>
61. Has anyone gotten Llama.cpp (or other) working using Intel iGPU,
https://www.reddit.com/r/LocalLLaMA/comments/1uur0ic/has_anyone_gotten_llamacpp_or_other_working_using/
62. How important is Vulkan back-end development? #590 - GitHub,
https://github.com/ikawrakow/ik_llama.cpp/discussions/590
63. PADDLEOCR. Introduction to OCR | by DhanushKumar - Medium,
<https://medium.com/@danushidk507/paddleocr-439a8d92fb1a>
64. PaddleOCR-VL: Boosting Multilingual Document Parsing via a 0.9B,
<https://ernie.baidu.com/blog/posts/paddleocr-vl/>
65. Character Error Rate (CER): A Friendly, No-Nonsense Guide,
<https://watercrawl.dev/blog/Character-Error-Rate>
66. Introducing Parrotlet-e and Eka-IndicMTEB: Bridging India's,
<https://www.eka.care/services/introducing-parrotlet-e-and-eka-indicmteb-bridging-indias-multilingual-healthcare-gap>
67. Context Precision - Ragas,
https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/context_precision/
68. Precision, Recall, Faithfulness, RAGAS Explained Clearly - YouTube,

https://www.youtube.com/watch?v=7_LTU0LA374